Development Prompt — Hockey Game Analyzer v1.0 Finalization Sprint

You are working on the `v1.0` release candidate of the Hockey Game Analyzer.

Repository:

`https://github.com/david-turnbull/Hockey-game-analyzer/tree/v1.0`

The project is already functionally close to release. This is **not** a feature-development sprint.

The objective is:

> **Finish, validate, package, and release Hockey Game Analyzer v1.0.**

Do not begin season analytics, public deployment, user accounts, trained xG, or other v1.1/v2 features.

This sprint should end with a repository that can confidently be tagged as:

`v1.0`

Current State

The application already includes:

- NHL API ingestion
- raw JSON caching
- SQLAlchemy relational database
- games, teams, players, events, shots, and shifts
- historical `GamePlayer` roster attribution
- game selector
- game overview
- scoring and penalty timelines
- interactive shot map
- Player Game Page
- centralized `OnIceService`
- half-open `[start, end)` shift semantics
- true 5v5 possession calculations
- Corsi and Fenwick
- forward-line combinations
- defence pairings

- experimental/prototype xG
- database integrity diagnostics
- automated tests
- real NHL fixture/regression tests
- GitHub Actions CI
- portfolio-oriented README
- service-layer separation

The major architecture and analytics work for v1.0 is finished.

This sprint should address only the remaining release-quality issues.

Primary Goal

At the end of this task, return one of:

```
READY TO TAG v1.0
READY TO TAG v1.0 WITH MINOR WARNINGS
NOT READY TO TAG v1.0
```

The preferred result is:

```
READY TO TAG v1.0
```

Do not create the Git tag or GitHub Release unless explicitly instructed.

Development Rules

1. Inspect the existing implementation before changing anything.
2. Do not redesign the application.
3. Do not add substantial new analytics.
4. Do not introduce unnecessary dependencies.
5. Preserve all existing functionality.
6. Every remaining correctness issue should have an explicit regression test.
7. Do not weaken tests to make CI pass.
8. Do not fabricate screenshots, metrics, or NHL data.
9. Do not claim something was tested unless it was actually executed.
10. Prefer small release-fix commits.
11. Keep analytical terminology precise.
12. Treat documentation and release metadata as part of the product.

Before making changes, provide a concise implementation plan identifying:

- files expected to change
- missing tests
- README/documentation changes
- CI/release changes
- anything that could block release

Then implement the work.

---

1. Complete Missing Regression Coverage

Review the existing test suite and release checklist.

Ensure there are explicit regression tests for every remaining v1.0 correctness fix.

At minimum add or verify tests for the following.

1.1 Empty Corsi/Fenwick Samples

Test:

```
CF = 0
CA = 0
=> CF% = None / undefined
```

and:

```
FF = 0
FA = 0
=> FF% = None / undefined
```

Also verify that a genuine zero remains zero:

```
CF = 0
CA = 5
=> CF% = 0.0
```

Do not confuse:

```
no observations
```

with:

```
0% share
```

1.2 UI Rendering of Undefined Possession

Verify that undefined possession values are rendered cleanly.

Expected UI behaviour should be something such as:

```
N/A
```

rather than:

```
None%
0%
50%
```

Test this at the template or route level where practical.

1.3 Average Shift Calculation

Add an explicit regression test proving anomalous shifts are excluded from:

- TOI
- valid shift count
- average shift duration

Example:

```
Shift 1 = 30 seconds
Shift 2 = 40 seconds
Shift 3 = zero-duration anomaly
```

Expected:

```
TOI = 70 seconds
Valid shifts = 2
Average shift = 35 seconds
```

Do not calculate:

```
70 / 3
```

1.4 Historical Team Attribution Fallback

Verify:

```
GamePlayer
    ↓
valid game-specific shift
    ↓
valid game-specific event
    ↓
unknown
```

is the historical-team resolution hierarchy.

Add a test where:

```
GamePlayer record is missing
Player.current_team_id is unrelated to historical game
Valid shift/event evidence exists
```

Expected:

The historical game-specific evidence is used.

Also test the case where no trustworthy evidence exists.

Expected:

```
Unknown / None
```

The code must never silently use `current_team_id` for historical attribution.

1.5 Line Change at Exact Shift Boundary

Test a line/pairing substitution using:

```
Player A = [600, 640)
Player B = [640, 680)
```

Expected:

```
639 -> Player A
640 -> Player B
```

Then verify the line combination generated at `640` contains the incoming player and not the outgoing player.

This should prove that `LineService` and `OnIceService` share identical semantics.

---

2. Audit Test Naming and Release Checklist References

Inspect:

```
release_checklist.md
```

The checklist must not reference tests or files that do not exist.

For every verified item:

- reference the correct test file/function where useful
- mark items complete only if actually verified
- remove stale references
- do not claim screenshots exist unless they do
- do not claim CI passed unless a workflow run actually passed

The release checklist should be auditable by another developer.

---

3. Final Analytics Terminology Pass

Review all user-facing terminology.

3.1 5v5 Possession

Any metrics specifically calculated as true 5v5 should be labelled:

```
5v5 Corsi
5v5 CF%
5v5 Fenwick
5v5 FF%
```

or equivalent.

Do not label them merely:

```
Even Strength
```

if 4v4 and 3v3 are excluded.

Review:

- game page
- player page
- tooltips
- tables
- README
- methodology notes

for consistency.

3.2 Prototype xG

The current xG feature is not a model trained on historical NHL outcomes.

User-facing wording should consistently use terminology such as:

```
Experimental xG
Prototype xG
Experimental expected-goal estimate
```

Do not describe it simply as:

```
our xG model
```

without qualification.

The README should explicitly state that coefficients are currently hand-selected.

---

4. Finish README Release Status

The README should not simultaneously describe v1.0 as both:

```
Release Candidate
```

and:

```
Current Release
```

Before tagging, use:

```
v1.0 Release Candidate
```

consistently.

After an actual release is created, this can be updated separately to:

```
Current Release: v1.0
```

Do not prematurely claim the release exists.

---

5. Finish Screenshot Handling

The README should not contain broken image references.

Preferred screenshots are:

```
Game selector
Game overview
Shot map
Player Game Page
Lines / pairings
```

Use:

```
docs/screenshots/
```

or an equivalent consistent repository location.

If screenshots already exist

Verify:

- files are committed
- links work
- images render on GitHub
- filenames are sensible
- captions explain what the user is seeing

If screenshots do not yet exist

Do not fabricate them.

Instead:

- create the directory if appropriate
- ensure README links are not broken
- leave a clearly documented release-note item such as:

```
Portfolio screenshots pending
```

Decide whether this is a technical release blocker or only a portfolio-presentation warning.

For a portfolio-focused v1.0, real screenshots are strongly preferred.

---

6. Verify Mermaid Architecture Rendering

Inspect the README architecture diagram.

Ensure it uses valid GitHub Markdown:

````
```mermaid

flowchart TD
    A[NHL API] --> B[Raw JSON Cache]
    B --> C[Transform and Validate]
    C --> D[SQLite / SQLAlchemy]
    D --> E[Service Layer]
    E --> F[Flask Routes / API]
    F --> G[Analytics UI]
````

````
Verify it renders rather than appearing as a plain code block.

Update the diagram to reflect the actual current architecture where useful:

```text
GameService
PlayerGameService
PossessionService
OnIceService
LineService
XGService
````

Do not overcomplicate the diagram.

---

7. Validate GitHub Actions

Inspect:

```
.github/workflows/tests.yml
```

Confirm CI runs for:

```
push -> v1.0
pull request -> main
```

and appropriate primary-development branches.

The CI workflow should:

```
checkout
setup Python
install dependencies
run pytest
```

from a clean environment.

Do not rely on:

- local SQLite database
- developer files
- uncommitted configuration
- local virtual environment

7.1 Run CI After Final Changes

The final release commit must receive a successful GitHub Actions run.

Do not call v1.0 ready merely because an earlier commit passed CI.

The exact commit intended for release should be green.

---

8. Run Full Local Test Suite

Run:

```
pytest
```

from a clean or representative environment.

Report actual:

```
Passed:
Failed:
Skipped:
```

Investigate any failures within v1.0 scope.

Do not hide or disable valid failing tests.

---

9. Run Database Diagnostics

Run the existing database integrity diagnostic.

Report actual counts for:

```
Games
Teams
Players
GamePlayer records
Events
Shot attempts
Shifts
```

Also report:

```
Orphan events
Orphan shots
Orphan shifts
Orphan GamePlayer records
Duplicate GamePlayer records
Historical team mismatches
Zero-duration shifts
Negative-duration shifts
Invalid manpower states
Unknown period types
Shots missing required shooters
Other warnings
```

Use actual database values.

Do not alter the diagnostic thresholds merely to produce PASS.

---

10. Fresh-Environment Smoke Test

Perform a clean-start validation.

The goal is to answer:

> Can someone clone this repository and run the software successfully?

Test the workflow documented in the README.

At minimum:

```
1. Create fresh Python environment
2. Install dependencies
3. Initialize database
4. Load representative NHL data
5. Run pytest
6. Start Flask application
7. Open main application
```

Smoke-test:

```
Game selector
Game overview
Shot map
Player Game Page
Possession
Lines
Defence pairings
Experimental xG
```

If anything requires an undocumented setup step, fix the documentation.

---

11. Dependency Review

Review the dependency file.

Remove:

- unused temporary development dependencies
- accidental local dependencies

- redundant packages

Do not aggressively upgrade packages as part of this release unless required for:

- security
- CI compatibility
- broken functionality

Avoid introducing release risk through unnecessary dependency upgrades.

---

12. Repository Hygiene

Before release, inspect the repository for files that should not be committed.

Examples:

```
__pycache__/
.pytest_cache/
venv/
.env
local databases
temporary exports
IDE files
large generated files
debug logs
```

Update `.gitignore` if necessary.

Do not remove legitimate cached NHL regression fixtures required by tests.

---

13. Verify No Secrets Are Present

Search the repository for:

```
API keys
passwords
tokens
connection strings
private URLs
```

```
credentials
.env contents
```

The NHL public-data pipeline may not require API credentials, but verify the repository regardless.

Do not print or expose secrets if discovered.

Report only that remediation is required and remove the secret from tracked code.

If a real secret has been committed historically, flag that it should be rotated.

---

14. Verify Release Metadata

Check:

- README
- version references
- release checklist
- branch wording
- package metadata if applicable
- application title/footer

They should agree on the status:

```
v1.0 Release Candidate
```

until the actual release/tag is created.

---

15. Create a v1.0 Release Summary

Prepare—but do not publish—a short release summary describing v1.0.

It should cover:

Core Features

- NHL data ingestion
- game analysis
- shot maps
- player analysis
- shift reconstruction

- historical rosters
- 5v5 possession
- line combinations
- defence pairings
- experimental xG

Engineering

- SQLAlchemy database
- raw-data caching
- data validation
- centralized on-ice service
- automated tests
- real NHL regression fixtures
- CI
- integrity diagnostics

Known Limitations

- public NHL data limitations
- second-level shift timing
- prototype xG not statistically fitted
- season-level analytics not yet implemented
- no production/public hosting in v1.0

Keep this concise enough for a GitHub Release description.

---

16. Confirm v1.0 Scope

The final v1.0 product should answer:

> **What happened in this NHL game, and which teams, players, lines, shots, and on-ice combinations influenced it?**

It should not attempt to answer season-level or predictive questions yet.

Do Not Add Before Release

Do not implement:

```
Casual / Intermediate / Seasoned modes
Season dashboard
```

```
Full-season ingestion expansion
Team season trends
Player season trends
Line season trends
Trained xG
XGBoost
WAR
RAPM
Predictions
Betting models
User accounts
Public deployment
PostgreSQL migration
Cloud scheduling
Video integration
Mobile app
Major UI redesign
```

These belong after v1.0.

Do not delay v1.0 by expanding scope.

---

v1.0 Definition of Done

The release candidate is ready when:

- [ ] all core existing features work
- [ ] all regression tests pass
- [ ] final commit has green CI
- [ ] historical team attribution is safe
- [ ] LineService uses centralized on-ice semantics
- [ ] `[start, end)` shift logic is consistent
- [ ] true 5v5 metrics are labelled correctly
- [ ] undefined CF% / FF% show N/A
- [ ] valid zero possession values remain 0%
- [ ] average shift calculations use valid shifts
- [ ] database diagnostics have been run
- [ ] clean-environment setup succeeds
- [ ] README setup instructions work
- [ ] Mermaid architecture renders
- [ ] release status wording is consistent
- [ ] no broken screenshot links exist
- [ ] screenshots are included or explicitly identified as a minor portfolio warning
- [ ] no secrets are committed
- [ ] repository hygiene is acceptable

- [ ] release checklist matches reality
- [ ] v1.0 release notes are prepared
- [ ] no new feature scope was introduced

Final Release Readiness Report

At completion, return:

1. Release Verdict

Exactly one:

```
READY TO TAG v1.0
READY TO TAG v1.0 WITH MINOR WARNINGS
NOT READY TO TAG v1.0
```

---

2. Tests

Report:

```
Passed:
Failed:
Skipped:
```

---

3. CI

Report:

```
Final commit tested:
Workflow status:
Python version:
```

---

4. Data Integrity

Summarize actual diagnostic results.

5. Remaining Warnings

Only list genuine remaining issues.

Classify each as:

```
BLOCKER
MINOR
POST-v1.0
```

---

6. Documentation

Confirm:

```
README setup tested
Mermaid rendering verified
Release status consistent
Screenshots status
Limitations documented
```

---

7. v1.0 Release Notes

Provide the prepared GitHub Release text.

If v1.0 Is Ready

Do not implement anything further.

Recommend the release sequence:

```
Final v1.0 commit
      ↓
GitHub Actions passes
      ↓
Merge to main
      ↓
```

```
CI passes on main
      ↓
Tag exact tested commit: v1.0
      ↓
Create GitHub Release
```

Then stop.

The next work should occur in a new development branch.

Post-v1.0 Direction — Do Not Implement Yet

After v1.0 is released, there are two logical development tracks.

v1.1 — Public User Experience

Focus on making the application usable by a wider audience:

```
Casual / Intermediate / Seasoned analysis modes
Plain-language explanations
Persistent browser preferences
Mobile/responsive polish
Feedback mechanism
Shareable game URLs
Public deployment preparation
```

v2.0 — Season Analytics

Focus on moving from:

```
What happened in this game?
```

to:

```
What has been happening across the season?
```

with:

```
Full-season ingestion
Team season dashboard
Player trends
Line/pairing trends
Season comparisons
Multi-season dataset
Trained xG preparation
```

Neither track should begin until v1.0 is formally closed.